

Tiny Second-hand Shopping Platform 개발 보고서

Tiny Second-hand Shopping Platform

시큐어 코딩 개발 보고서

White Hat School (WHS39)

항목	내용
이름	이서영
과정	White Hat School WHS39
제출 과제	Tiny Second-hand Shopping Platform (Secure Coding)
기술 스택	Python 3.12, FastAPI, Jinja2, SQLAlchemy, SQLite
GitHub 저장소	https://github.com/exlink0807/tiny-secondhand-shop
제출일	2026년 7월 23일

들어가며

이번 과제는 중고거래 플랫폼을 처음부터 끝까지 직접 설계하고 구현하는 것이었다. 그냥 기능만 돌아가게 만드는 게 아니라, 요구사항 분석부터 설계, 구현, 테스트, 유지보수 계획까지 개발 전 과정을 문서로 남기고 그 안에서 보안을 어떻게 고민했는지 보여주는 게 핵심이라고 생각했다.

처음에는 취약점을 일부러 넣어놓고 나중에 고치는 방식(vulnerable-by-design)으로 할지, 아니면 처음부터 안전하게 짜고 리뷰 형태로 정리할지 고민했는데, 실제 실무에서는 후자 쪽이 더 현실적인 개발 방식이라고 판단해서 처음부터 보안을 염두에 두고 구현하는 쪽을 택했다. 대신 코드를 짜면서 “이렇게 하면 뚫리지 않을까?”를 계속 스스로 점검하고, 그 과정에서 실제로 방향을 바꾼 부분들을 솔직하게 기록해두었다.

개발 과정에서 Claude(Anthropic)를 적극적으로 사용했다. 설계 문서 초안 작성, 코드 구현, 테스트 케이스 작성, 그리고 무엇보다 “이 부분에 이런 취약점이 있을 수 있다”는 지적을 받고 설계를 고치는 데 많은 도움을 받았다. 다만 AI가 제안한 내용을 그대로 쓰기보다는, 왜 그렇게 해야 하는지 이해하고 직접 검증(테스트 실행, 클린 환경 재현)한 뒤에 반영했다.

1. 요구사항 분석 (Requirements Analysis)

1.1 프로젝트 개요

- 프로젝트명: Tiny Second-hand Shopping Platform
- 목적: 중고거래가 가능한 웹 플랫폼을 안전하게(Secure Coding) 설계·구현
- 대상 사용자: 일반 회원(구매자/판매자 겸용), 관리자(운영자)

1.2 기능 요구사항 (제시된 최소 요구사항 기반)

ID	요구사항	상세 기능
FR-01	회원가입/인증	회원가입, 로그인/로그아웃, 세션 관리
FR-02	상품 등록/조회	상품 등록(이미지 포함), 상세조회, 목록조회, 수정/삭제(본인만)
FR-03	사용자 간 소통	상품 기준 1:1 채팅(문의/거래 대화)
FR-04	악성 유저/상품 차단	신고 기능(유저/상품), 관리자 검토 후 차단(정지) 처리

FR-05	유저 간 송금	포인트(잔액) 기반 송금, 거래 내역 조회
FR-06	상품 검색	제목/카테고리/가격 범위 기반 검색
FR-07	관리자 기능	회원 관리(정지/해제), 상품 관리(숨김/삭제), 신고 처리, 거래 로그 조회, 감사 로그

1.3 사용자 스토리 (일부)

- 사용자로서, 회원가입 후 로그인하여 내 상품을 등록하고 싶다.
- 구매자로서, 판매자에게 채팅으로 문의하고 안전하게 포인트를 송금하고 싶다.
- 사용자로서, 악성 판매자/허위 매물을 신고하고 싶다.
- 관리자로서, 신고된 사용자/상품을 검토하여 정지·삭제 처리하고 싶다.
- 관리자로서, 전체 회원/상품/거래를 모니터링하고 싶다.

1.4 비기능 요구사항 (보안 중심)

ID	항목	내용
NFR-01	인증/인가	비밀번호 해시 저장, 세션 하이재킹 방지, 모든 API에 권한 검증(IDOR 방지)
NFR-02	입력 검증	서버 측 검증(Pydantic), SQL Injection/XSS 방지
NFR-03	파일 업로드 안전성	확장자/시그니처 검증, 저장 파일명 랜덤화, 실행권한 제거
NFR-04	트랜잭션 무결성	송금 시 원자적 처리(동시성/이중 지출 방지), 잔액 음수 방지
NFR-05	CSRF 방지	상태 변경 요청에 CSRF 토큰 검증
NFR-06	로깅/감사	관리자 행위 및 주요 이벤트 로깅(민감정보 제외)
NFR-07	에러 처리	운영 모드에서 스택트레이스/내부 정보 미노출
NFR-08	전송 보안	보안 헤더 적용(CSP, X-Frame-Options 등), 쿠키 HttpOnly/SameSite
NFR-09	Rate Limiting	로그인 등 브루트포스 방지용 시도 제한

1.5 범위 외 (Out of Scope)

- 실제 PG(결제대행)사 연동 결제 — 포인트는 데모용 가상 잔액으로 대체 (회원가입 시 기본 지급 + 관리자 지급 기능으로 대체)
- 실시간 알림(웹소켓 푸시) — 폴링 기반 채팅으로 단순화 가능(구현 여유 시 WebSocket 적용) # 2. 시스템 설계 (System Design)

2.1 아키텍처

```
[Browser] --HTTPS--> [FastAPI (Uvicorn)]
    |-- Jinja2 Templates (SSR, autoescape)
    |-- Session Middleware (서명된 쿠키, HttpOnly/SameSite=Strict)
    |-- SQLAlchemy ORM ---> [SQLite DB]
    |-- Static/Upload 파일 서빙 (검증된 파일만)
```

- 서버사이드 렌더링(Jinja2) 기반의 단순 구조. 클라이언트에 토큰(JWT)을 저장하지 않고, 서버 서명 세션 쿠키 사용 → XSS로 인한 토큰 탈취 표면 축소.
- 모든 상태 변경 요청(POST/PUT/DELETE)에 CSRF 토큰 필수.

2.2 ERD (개체-관계 모델)

```
User
- id (PK)
```

```

- username (unique)
- email (unique)
- password_hash
- role (enum: user, admin)
- status (enum: active, suspended)
- balance (integer, 원 단위, >= 0)
- created_at

Product
- id (PK)
- seller_id (FK -> User.id)
- title
- description
- price (integer)
- category
- status (enum: available, sold, hidden_by_admin, deleted)
- image_path (nullable)
- created_at

Conversation
- id (PK)
- product_id (FK -> Product.id)
- buyer_id (FK -> User.id)
- seller_id (FK -> User.id)
- created_at
- unique(product_id, buyer_id)

Message
- id (PK)
- conversation_id (FK -> Conversation.id)
- sender_id (FK -> User.id)
- content
- created_at

Report
- id (PK)
- reporter_id (FK -> User.id)
- target_type (enum: user, product)
- target_id (int)
- reason
- status (enum: pending, reviewed, actioned, dismissed)
- created_at
- reviewed_by (FK -> User.id, nullable)
- reviewed_at (nullable)

Transaction (송금)
- id (PK)
- sender_id (FK -> User.id)
- receiver_id (FK -> User.id)
- amount (integer, > 0)
- memo
- created_at

AuditLog (관리자 행위 기록)
- id (PK)
- admin_id (FK -> User.id)
- action (예: SUSPEND_USER, HIDE_PRODUCT, RESOLVE_REPORT, GRANT_BALANCE)
- target_type
- target_id
- detail
- created_at

```

관계: - User 1 — N Product (seller) - Product 1 — N Conversation - Conversation 1 — N Message - User 1 — N Report (reporter) - User 1 — N Transaction (sender), 1 — N Transaction (receiver)

2.3 API 설계 (요약)

메서드	경로	설명	권한

GET/POST	/register	회원가입	비로그인
GET/POST	/login	로그인	비로그인
POST	/logout	로그아웃	로그인
GET	/products	상품 목록/검색	전체
GET	/products/{id}	상품 상세	전체
GET/POST	/products/new	상품 등록	로그인
GET/POST	/products/{id}/edit	상품 수정	소유자
POST	/products/{id}/delete	상품 삭제	소유자/관리자
GET	/chat	대화 목록	로그인
GET/POST	/chat/{conversation_id}	채팅방	대화 참여자만
POST	/products/{id}/chat/start	채팅 시작(구매자)	로그인
POST	/reports/new	신고 등록	로그인
GET/POST	/wallet	잔액 확인 및 송금	로그인
GET	/admin	관리자 대시보드	관리자
POST	/admin/users/{id}/suspend	유저 정지/해제	관리자
POST	/admin/products/{id}/hide	상품 숨김/복구	관리자
POST	/admin/reports/{id}/resolve	신고 처리	관리자
POST	/admin/users/{id}/grant	데모용 잔액 지급	관리자

모든 “권한: 소유자/참여자/관리자” 엔드포인트는 서버에서 `현재 로그인 사용자 == 리소스 소유자 or role==admin` 을 반드시 검증 (IDOR 방지).

2.4 위협 모델링 (STRIDE 요약)

위협 유형	시나리오	대응
Spoofing	세션 쿠키 위조/탈취	서명된 세션(itsdangerous), HttpOnly+Secure+SameSite, 세션 만료
Tampering	상품 가격/타인 채팅 조작	서버측 소유자 검증, ORM 파라미터 바인딩
Repudiation	관리자 조치 부인	AuditLog 기록
Information Disclosure	타 유저 정보/채팅 열람(IDOR), 에러 스택 노출	인가 검사, 프로덕션 에러 핸들러
Denial of Service	로그인 브루트포스, 대량 요청	로그인 시도 제한(rate limit), 잠금
Elevation of Privilege	일반 유저가 관리자 API 접근	role 기반 의존성 검사(require_admin)

2.5 보안 설계 원칙 (구현 전 확정)

- 비밀번호: bcrypt(passlib)로 해시, 원문 저장/로그 금지
- DB 접근: SQLAlchemy ORM만 사용, raw SQL 문자열 결합 금지
- 출력: Jinja2 autoescape 유지 (수동 |safe 사용 금지)
- 인가: 모든 수정/조회 라우트에서 소유권·역할 검사를 의존성(Depends)으로 강제
- 파일 업로드: 확장자 화이트리스트 + 매직바이트 검증 + UUID 파일명 + 정적 서빙 경로 분리
- 송금: DB 트랜잭션 + row lock(또는 SQLite 특성상 단일 트랜잭션 커밋)으로 동시성 이슈 방지, 잔액 검증
- CSRF: 폼마다 hidden token, 서버에서 세션 저장 토큰과 대조
- 보안 헤더 미들웨어: CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy
- 로깅: 실패 로그인 시도, 관리자 액션만 기록. 비밀번호/세션토큰 로깅 금지 # 3. 보안 체크리스트 및 취약점 리뷰 (OWASP Top 10:2021 기준)

“먼저 취약점을 심고 나중에 고치는” 방식 대신, 설계 단계부터 안전하게 짜는 쪽으로 진행했다. 그래서 아래 표는 “발견 → 수정” 기록이라기보다, OWASP Top 10 항목별로 어떤 위험을 검토했고 어떤 방어 로직을 넣었으며 어떤 테스트로 확인했는지를 정리한 리뷰에 가깝다. 실제로 코드를 짜다가 방향을 바꾼 부분은 3.2에 따로 적어두었다.

3.1 OWASP Top 10:2021 매핑

#	항목	이 프로젝트에서의 위험	구현한 대응	검증 테스트
A01	Broken Access Control	타인의 상품 수정/삭제(IDOR), 타인 채팅방 열람, 일반 유저의 관리자 기능 접근	모든 라우트에서 <code>require_login / require_admin</code> 의존성 강제, 리소스 조회 후 <code>seller_id / 참여자 id</code> 와 현재 로그인 유저 비교	<code>test_authorization.py</code> (403 확인 6건)
A02	Cryptographic Failures	비밀번호 평문 저장/노출	bcrypt 해시 저장(passlib), 평문 비밀번호 로깅 금지, 세션은 서버 서명 쿠키 사용(JWT를 localStorage에 두지 않음)	코드 리뷰 (security.py)
A03	Injection (SQL/XSS)	검색어를 통한 SQL Injection, 채팅/상품명 등을 통한 Stored XSS	SQLAlchemy ORM만 사용 (raw SQL 미사용), Jinja2 autoescape 유지(<code> safe</code> 미사용)	<code>test_product_search_is_safe_against_sql_injection_like_input</code> , <code>test_stored_xss_payload_is_escaped_in_product_title</code>
A04	Insecure Design	송금 시 잔액 음수/이중지출, 회원가입 시 과도한 포인트 지급 등 비즈니스 로직 결함	DB <code>CheckConstraint(balance >= 0)</code> , 트랜잭션 단위 잔액 재검증, 관리자 지급 한도(1회 최대 100만원)	<code>test_wallet.py</code> (잔액부족/음수금액/한도), <code>test_admin_grant_balance_rejects_excessive_amount</code>
A05	Security Misconfiguration	운영 환경에서 상세 에러/스택트레이스 노출, 보안 헤더 미설정	전역 예외 핸들러가 <code>DEBUG=false</code> 일 때 내부 오류 은닉, CSP/X-Frame-Options/X-Content-Type-Options 등 보안 헤더 미들웨어 적용	수동 curl 헤더 확인 (§ 4 테스트 로그)
A06	Vulnerable and Outdated Components	알려진 취약점이 있는 패키지 버전 사용	requirements.txt에 버전 고정, 정기적 <code>pip list --outdated</code> 점검 권장 (README 참고)	수동 점검
A07	Identification and Authentication Failures	브루트포스 로그인, 세션 고정(Session Fixation), 계정 존재 여부 노출	로그인 5회 실패 시 5분 잠금, 로그인 성공 시 세션 재발급(clear 후 재구성), 로그인 실패 메시지를 계정 존재 여부와 무관하게 동일하게 처리	<code>test_login_rate_limited_after_repeated_failures</code> , <code>test_login_wrong_password_generic_message</code> , <code>test_login_nonexistent_user_same_generic_message</code>
A08	Software and Data Integrity Failures	업로드 파일을 통한 악성 코드 실행(확장자 위장)	확장자 화이트리스트 + 매직바이트(실제 파일 시그니처) 검증 + 저장 파일명 UUID 재생성	<code>test_upload_rejects_non_image_extension</code> , <code>test_upload_rejects_fake_extension_with_wrong_content</code> , <code>test_upload_accepts_valid_id_png</code>
A09	Security Logging and Monitoring Failures	관리자 행위 부인/추적 불가	<code>AuditLog</code> 테이블에 관리자 행위(정지, 숨김, 신고처리, 지급) 기록, 실패 로그인 로그(향후 확장 가능)	<code>test_admin.py</code> (조치 후 대시보드 로그 노출 확인)

A10	Server-Side Request Forgery (SSRF)	본 앱은 사용자 입력을 기반으로 서버가 외부 URL을 요청하는 기능이 없음(해당 없음)	해당 기능 미구현으로 공격면 자체가 없음	-
-----	------------------------------------	--	------------------------	---

또한 OWASP Top 10에 명시적으로 없지만 검토한 항목:

위험	대응
CSRF	모든 상태 변경 POST 요청에 세션 저장 토큰과 폼 토큰을 <code>secrets.compare_digest</code> 로 비교 검증
클릭재킹	<code>X-Frame-Options: DENY</code> , <code>CSP frame-ancestors 'none'</code>
MIME 스니핑	<code>X-Content-Type-Options: nosniff</code>
계정 열거(Enumeration)	회원가입 시 아이디/이메일 중복만 안내(부득이 노출), 로그인 실패 시에는 동일 메시지로 통일
대용량 파일 업로드(DoS)	업로드 크기 5MB 제한
자기 자신에게 송금 등 논리적 이상 흐름	서버측에서 별도 검증 후 차단

3.2 코드 리뷰 과정에서 조정한 사항

설계와 구현을 진행하며 실제로 방향을 수정한 부분들입니다.

- 세션 저장 방식:** 처음에는 클라이언트 저장에 간편한 JWT를 고려했으나, 브라우저(localStorage)에 저장 시 XSS 한 번으로 토큰이 탈취될 수 있어 서버 서명 쿠키(HttpOnly) 기반 세션으로 설계를 변경함.
- 로그인 실패 메시지:** 최초에는 “존재하지 않는 아이디입니다”/“비밀번호가 틀렸습니다”로 구분하려 했으나, 이는 계정 존재 여부를 공격자에게 알려주는 사용자 열거(User Enumeration) 취약점이 되므로 두 경우 모두 동일한 문구로 통일함.
- 파일 업로드 검증:** 확장자 검사만으로는 .jpg 로 이름만 바꾼 스크립트 파일 업로드를 막을 수 없어, 매직바이트(파일 시그니처) 검증을 추가함. `python-magic` (libmagic) 사용이 불가능한 배포 환경을 고려해 시그니처 하드코딩 fallback도 함께 구현함.
- 송금 동시성:** 단순히 “잔액 조회 → 차감”으로 구현할 경우 동시 요청 시 이중 지출 가능성이 있어, 같은 DB 트랜잭션 내에서 최신 잔액을 재조회 후 커밋하는 방식으로 조정하고, `CheckConstraint(balance >= 0)` 를 DB 레벨에서도 강제함. (다중 워커/분산 배포 시에는 PostgreSQL 등 row-lock 지원 DB로 전환 필요 — README/설계문서에 명시)
- 비밀번호 해시 라이브러리 버전 충돌:** `passlib[bcrypt]` 최신 `bcrypt` (5.x)와 호환성 문제(72바이트 초과 등 내부 버그 감지 로직 오류)가 있어 `bcrypt==4.0.1` 로 버전을 고정함. (기능 버그였으나 인증 핵심 로직이라 보안 체크리스트에 기록)
- 관리자 초기 계정:** 하드코딩된 고정 비밀번호로 관리자 계정을 생성하면 소스가 공개(GitHub public)되었을 때 그대로 노출되므로, 환경변수 미설정 시 매 최초 실행마다 랜덤 비밀번호를 생성해 최초 1회만 콘솔에 출력하도록 변경함.
- SECRET_KEY 고정 위험:** 세션 서명 키를 코드에 하드코딩하면 GitHub 공개 저장소에 노출되어 세션 위조가 가능해지므로, 환경변수 미설정 시 매 실행마다 랜덤 키를 생성하도록 처리(단, 이 경우 서버 재시작 시 기존 세션은 무효화됨 — 운영 배포 시에는 반드시 `SECRET_KEY` 환경변수 고정 필요, README에 명시).

3.3 테스트 실행 결과 요약

```
tests/ 31 passed (pytest, FastAPI TestClient 기반)
tests/smoke_test.py 18/18 통과 (실제 실행 서버 대상 end-to-end 시나리오)
```

실행 방법은 README.md의 “테스트 실행” 항목 참고. # 4. 유지보수 계획 (Maintenance)

4.1 의존성 관리

- `requirements.txt` 에 모든 패키지 버전을 고정(pin)하여 배포 환경 재현성을 확보.
- 월 1회 `pip list --outdated` 로 알려진 취약점(CVE)이 있는 패키지 확인 후 마이너 버전 업데이트.
- 특히 `passlib`, `bcrypt`, `starlette`, `fastapi` 는 인증/세션 관련 핵심 의존성이므로 CVE 공지를 우선 확인.

4.2 배포 환경 전환 시 체크리스트

- `DATABASE_URL` 을 PostgreSQL 등으로 전환 시 `wallet.py` 의 동시성 처리 로직 (§ 2.5, § 3.2-4)을 `SELECT ... FOR UPDATE` 기반으

로 재검증.

- SECRET_KEY, ADMIN_PASSWORD 를 환경변수로 반드시 고정 지정(미지정 시 재시작마다 랜덤 값으로 세션 무효화/관리자 비밀번호 변경됨).
- 리버스 프록시(Nginx 등) 뒤에서 HTTPS로 서비스할 경우 HTTPS_ONLY=true 로 세션 쿠키 Secure 플래그 활성화.

4.3 로깅/모니터링

- 현재 애플리케이션 로고는 표준 logging 모듈로 콘솔 출력. 운영 배포 시 파일 로테이션(TimedRotatingFileHandler) 또는 중앙 로그 수집(ELK 등) 연동 권장.
- AuditLog 테이블은 삭제되지 않고 계속 누적되므로, 일정 주기로 아카이빙 정책 수립 필요.
- 5xx 에러 발생 시 알림(Slack/이메일 등) 연동은 향후 개선 과제로 분류.

4.4 백업

- SQLite 파일(app.db) 기준 매일 1회 스냅샷 백업 권장. PostgreSQL 전환 시 pg_dump 기반 정기 백업으로 전환.

4.5 기능 확장 시 보안 회귀 방지

- 새 라우트 추가 시 아래 3가지를 반드시 리뷰:
 1. 로그인에 필요한가? (require_login / require_admin 의존성 부착 여부)
 2. 리소스 소유자 검증이 필요한가? (IDOR 방지)
 3. 상태를 변경하는 POST/PUT/DELETE라면 CSRF 토큰 검증이 포함되었는가?
- tests/ 하위에 최소 1개 이상의 인가/입력검증 테스트를 추가하고 CI(또는 PR 전 로컬 pytest)에서 통과를 확인 후 병합.

4.6 버전 관리 및 브랜치 전략

- main 브랜치는 항상 배포 가능한 상태 유지.
- 기능 단위로 브랜치를 분리(feature/xxx)하고, 병합 전 pytest 전체 통과를 필수 조건으로 함. # 5. 테스트 상에 실행 결과

5.1 pytest 단위/통합 테스트 (총 31개)

FastAPI TestClient 와 격리된 임시 SQLite DB를 사용해 인증, 인가(IDOR), CSRF, 파일 업로드 검증, XSS 이스케이프, 송금 로직, 관리자 기능을 검증했다. 클린 가상환경(python -m venv + pip install -r requirements.txt)에서도 동일하게 재현되는지 확인했다.

```
tests/test_admin.py::test_admin_can_suspend_and_unsuspend_user PASSED
tests/test_admin.py::test_admin_cannot_suspend_admin_account PASSED
tests/test_admin.py::test_admin_can_hide_product PASSED
tests/test_admin.py::test_admin_grant_balance PASSED
tests/test_admin.py::test_admin_grant_balance_rejects_excessive_amount PASSED
tests/test_auth.py::test_register_success PASSED
tests/test_auth.py::test_register_rejects_weak_password PASSED
tests/test_auth.py::test_register_rejects_invalid_username PASSED
tests/test_auth.py::test_register_duplicate_username_rejected PASSED
tests/test_auth.py::test_login_success PASSED
tests/test_auth.py::test_login_wrong_password_generic_message PASSED
tests/test_auth.py::test_login_nonexistent_user_same_generic_message PASSED
tests/test_auth.py::test_login_rate_limited_after_repeated_failures PASSED
tests/test_auth.py::test_logout_clears_session PASSED
tests/test_authorization.py::test_anonymous_cannot_create_product PASSED
tests/test_authorization.py::test_non_owner_cannot_edit_product PASSED
tests/test_authorization.py::test_non_owner_cannot_delete_product PASSED
tests/test_authorization.py::test_only_conversation_participants_can_view_chat PASSED
tests/test_authorization.py::test_admin_routes_require_admin_role PASSED
tests/test_authorization.py::test_admin_login_and_dashboard_access PASSED
tests/test_products.py::test_product_creation_requires_csrf_token PASSED
tests/test_products.py::test_product_search_is_safe_against_sql_injection_like_input PASSED
tests/test_products.py::test_upload_rejects_non_image_extension PASSED
tests/test_products.py::test_upload_rejects_fake_extension_with_wrong_content PASSED
tests/test_products.py::test_upload_accepts_valid_png PASSED
tests/test_products.py::test_stored_xss_payload_is_escaped_in_product_title PASSED
tests/test_wallet.py::test_transfer_success_updates_balances PASSED
tests/test_wallet.py::test_transfer_rejects_insufficient_balance PASSED
```

```
tests/test_wallet.py::test_transfer_rejects_negative_or_zero_amount PASSED
tests/test_wallet.py::test_transfer_rejects_self_transfer PASSED
tests/test_wallet.py::test_transfer_rejects_nonexistent_receiver PASSED

===== 31 passed in 18.83s =====
```

5.2 엔드투엔드 스모크 테스트 (실제 서버 구동 상태, 18개 시나리오)

tests/smoke_test.py 는 실제로 uvicorn 서버를 띄운 상태에서 requests 세션으로 회원가입부터 관리자 기능까지 실제 사용자의 흐름을 그대로 재현해 검증한다.

```
[OK] 판매자 회원가입
[OK] 구매자 회원가입
[OK] 중복 아이디 회원가입 거부
[OK] 판매자 로그인
[OK] 구매자 로그인
[OK] 로그인 실패 메시지 동일(계정 존재 비노출)
[OK] 잘못된 CSRF 토큰 거부
[OK] 상품 등록
[OK] 상품 ID 파싱
[OK] 상품 검색
[OK] 타인 상품 수정 접근 차단 (403)
[OK] 채팅 시작
[OK] 메시지 전송
[OK] 제3자 채팅방 접근 차단 (403)
[OK] 송금 성공
[OK] 잔액 초과 송금 거부
[OK] 신고 접수
[OK] 비로그인 관리자 접근 -> 로그인 리다이렉트
[OK] 일반 유저 관리자 접근 차단 (403)
```

모든 스모크 테스트 통과!

5.3 수동 검증 (보안 헤더, 에러 처리)

```
$ curl -s -D - -o /dev/null http://127.0.0.1:8000/products
HTTP/1.1 200 OK
x-content-type-options: nosniff
x-frame-options: DENY
referrer-policy: same-origin
content-security-policy: default-src 'self'; img-src 'self' data:; style-src 'self'
'unsafe-inline'; script-src 'self'; frame-ancestors 'none'; base-uri 'self'; form-action 'self'
permissions-policy: geolocation=(), microphone=(), camera=()

$ curl -s http://127.0.0.1:8000/products/99999
(존재하지 않는 상품 ID) -> 404, "요청하신 페이지를 찾을 수 없습니다" (내부 스택트레이스 미노출)

$ curl -s "http://127.0.0.1:8000/products?q=%27%200R%20%271%27%3D%271"
-> 200 정상 응답 (SQL Injection 유사 입력이 검색어 리터럴로만 처리됨, 에러 미발생)
```

5.4 체크리스트 (제출 전 최종 확인)

- ☒ 회원가입/로그인/로그아웃 정상 동작
- ☒ 상품 등록/조회/검색/수정/삭제 정상 동작 및 소유자 검증
- ☒ 채팅(1:1) 정상 동작 및 참여자 검증
- ☒ 신고 → 관리자 처리 플로우 정상 동작
- ☒ 송금 정상 동작, 잔액 부족/음수/자기송금 거부
- ☒ 관리자 기능(정지/숨김/신고처리/지급/감사로그) 정상 동작
- ☒ 비로그인/권한없는 사용자의 보호된 라우트 접근 차단
- ☒ CSRF 토큰 누락/위조 시 요청 거부
- ☒ 악성 파일 업로드(확장자 위장 포함) 거부

- ☑ Stored XSS 페이로드가 출력 시 이스케이프됨
- ☑ 클린 가상환경에서 requirements.txt 기준 설치 및 테스트 재현 확인
- ☑ GitHub public 저장소 업로드 및 README 작성 완료 # 6. 저장소 및 실행 방법

6.1 GitHub 저장소

- URL: <https://github.com/exlink0807/tiny-secondhand-shop> (public)
- README.md 에 환경 설정 및 실행 방법을 명시함.

6.2 요약 실행 방법

```
git clone https://github.com/exlink0807/tiny-secondhand-shop.git
cd tiny-secondhand-shop
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
# http://localhost:8000 접속
# 최초 실행 시 콘솔에 출력되는 admin 계정 비밀번호로 관리자 로그인 가능
```

테스트 실행:

```
pip install pytest httpx
pytest tests/ -v
```

6.3 프로젝트 구조 요약

app/	FastAPI 애플리케이션 (라우터, 모델, 보안 유틸)
templates/	Jinja2 서버사이드 렌더링 템플릿
static/	CSS, 업로드 이미지
docs/	요구사항/설계/보안체크리스트/유지보수 문서 (본 보고서의 원본)
tests/	pytest 단위테스트 + 스모크 테스트

7. 결론 및 회고

요구사항에 나와 있던 8가지 기능 — 회원가입, 상품 등록/조회, 유저 간 소통, 악성 유저/상품 차단, 유저 간 송금, 상품 검색, 관리자 기능 — 은 전부 구현했고, 각 기능을 만들 때마다 “여기서 누가 악의적으로 접근하면 어떻게 될까”를 먼저 생각하고 코드를 짜려고 했다.

돌이켜보면 가장 신경 쓴 부분은 인가(authorization) 쪽이었다. 상품 수정/삭제, 채팅방, 관리자 페이지처럼 “내 것”과 “남의 것”을 구분해야 하는 곳이 많았는데, 이런 IDOR류 취약점은 기능이 겹으로는 멀쩡하게 동작해서 놓치기 쉽다는 걸 이번에 다시 느꼈다. 그래서 테스트를 짤 때도 정상 동작 확인보다 “이 사람이 이걸 접근하면 안 되는데 접근이 되나?”를 확인하는 테스트를 더 많이 넣었다.

몇 가지는 만들면서 실제로 방향을 바꿨다. 로그인 실패 메시지를 처음에는 “아이디가 없습니다” / “비밀번호가 틀렸습니다”로 나눠서 보여주려고 했는데, 이러면 공격자가 아이디 존재 여부를 알아낼 수 있다는 걸 짚어주셔서(Claude와 대화하면서) 똑같은 메시지로 통일했다. 파일 업로드도 처음엔 확장자만 검사했는데, 확장자만 .png 로 바꾸면 통과된다는 걸 알고 매직바이트 검증을 추가했다. 송금 기능은 동시에 여러 번 요청이 오면 잔액이 꼬일 수 있다는 점을 미처 생각 못 하고 있었는데, 이 부분도 DB 트랜잭션과 제약조건으로 다시 잡았다.

사실 보안이랑 상관없는 곳에서도 삼진을 줬다. passlib 이 최신 bcrypt (5.x)랑 안 맞아서 비밀번호 해싱 자체가 에러를 뱉는 걸 한참 찾다가 bcrypt==4.0.1 로 버전을 고정하고서야 해결됐다. 이런 건 취약점이라기보다 그냥 의존성 버전 문제였지만, 실무에서도 흔히 겪는 일이라 체크리스트 문서에 같이 남겨두었다.

남은 과제도 분명히 있다. SQLite는 지금 구조(단일 프로세스)에서는 문제없지만 여러 워커로 배포하면 송금 동시성 로직을 다시 봐야 하고, 채팅도 지금은 새로고침 기반이라 실시간성이 필요하면 웹소켓으로 바뀌어야 한다. 결제도 실제 PG 연동 없이 가상 포인트로 대체한 부분이라 나중에 다르게 확장될 수 있는 지점이다. 이런 부분들은 docs/04_maintenance.md 에 정리해두었다.